EVAN BAESLER
October 25, 2026

EE Design 1 Technical Report
Analog Filter Design

Introduction

The goal of this assignment was to create a filter that can transform a square wave of a set frequency into the sinusoidal equivalent, proving that a square wave is simply the sum of the harmonic sinusoidal waves. Additional scope required us to have a near 80dB/decade roll off, and to find a proper frequency of attenuation to keep our 1st harmonic whilst attenuating the rest.

Methods

Pre-Design Filter Verification

The filter used in this assignment was a Butterworth, for the simple reason that it offers a smooth curve for our output Bode plot, without any ripples affecting our passband and stopband. This was designed with a python script listed at the appendix of this document, the results for a 1.65kHz cutoff through an Analog filter are shown below:

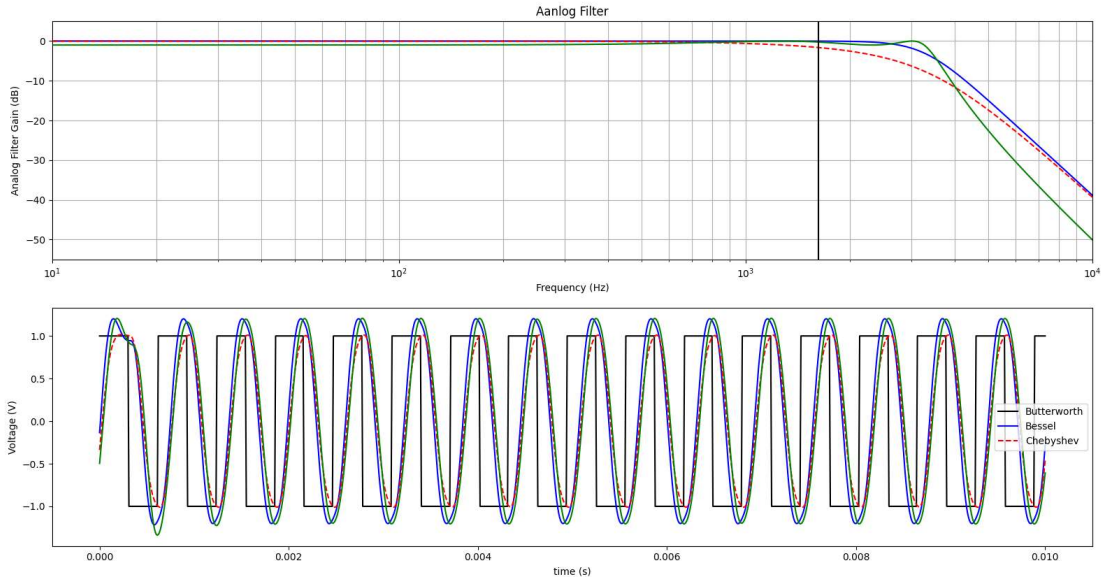


Figure 1: Python Simulated Filter Bode & Transient

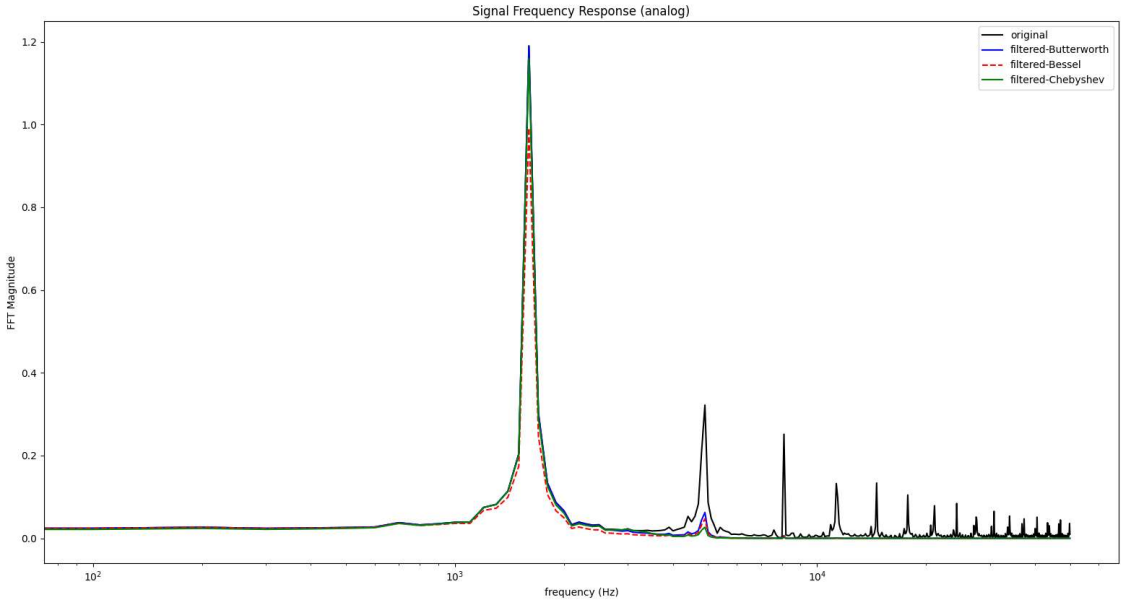


Figure 2: Python Simulated Filter Spectra Chart

Filter Design

This filter was implemented utilizing the Analog Devices filter design tool, located at:

- https://tools.analog.com/en/filterwizard/
    - 1.65kHz -3dB
    - 16.5kHz -80dB (~-80dB/dec or -80dB per 10x in frequency)
    - As implemented with RC sizing to aim for easily accessible capacitors (~100nF)
    - Sallen-Key architectures

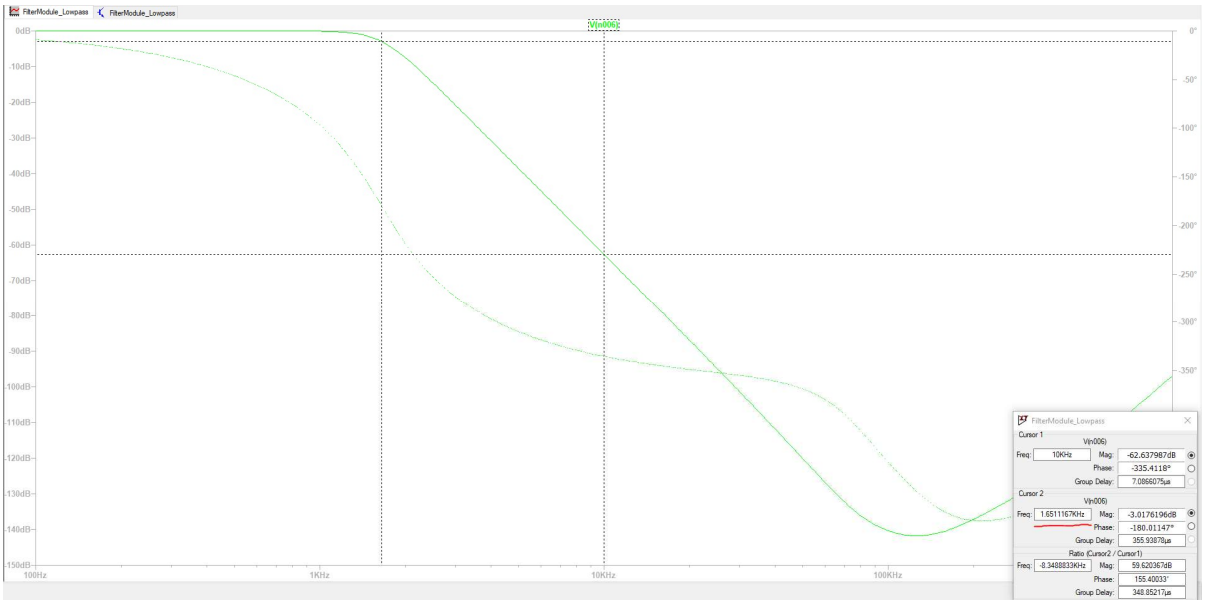


Figure 3: Practical Filter Bode (AC) Plot

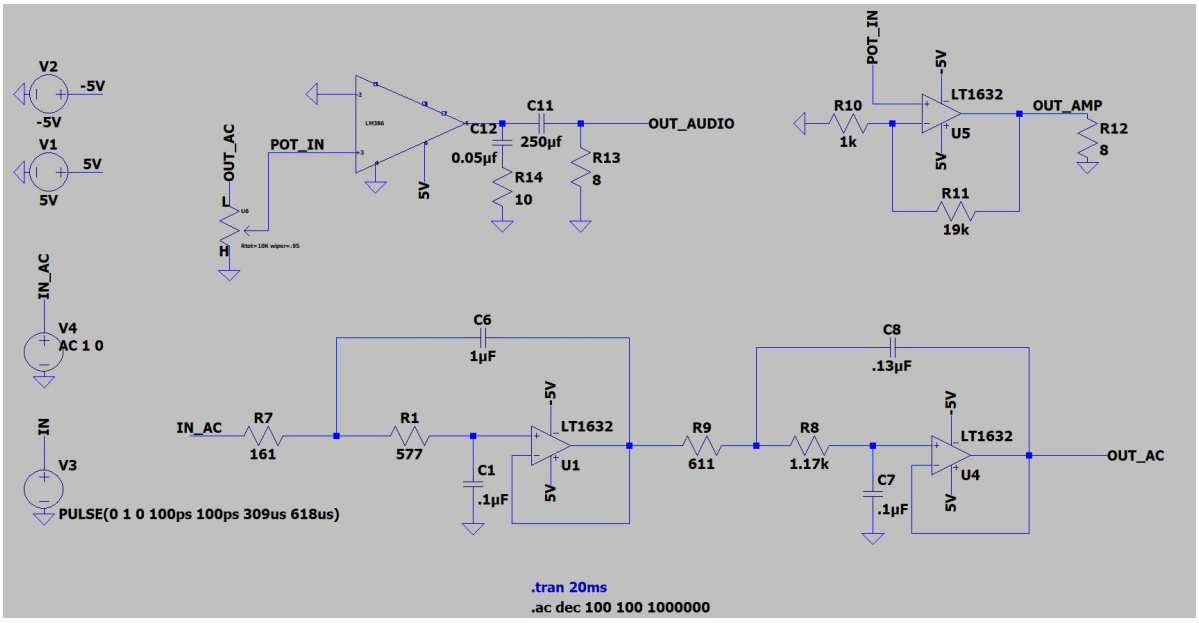


Figure 4: Filter & Amp Design

Transfer Function Derivation Form

Utilizing the information that these are Sallen-Key architectures, we can easily find (or hand-derive) the transfer function H(s).

As found on Wikipedia (1):

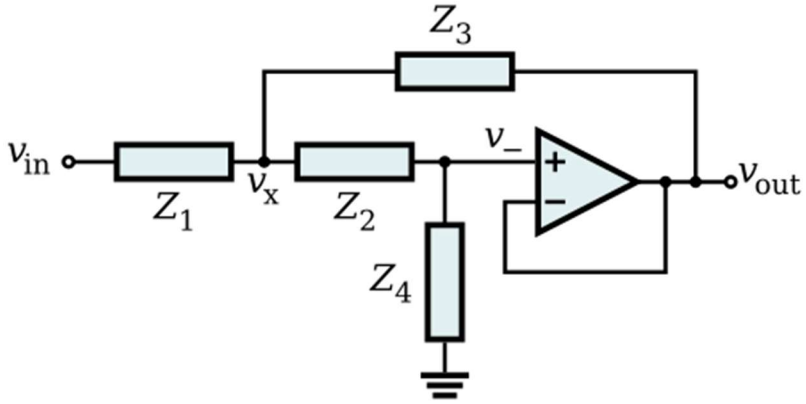


Figure 5: Sallen-Key Filter Architecture

$$\frac{V_o}{V_i} = \frac{Z_3 Z_4}{Z_1 Z_2 + Z_3(Z_1 + Z_2) + Z_3 Z_4}$$

Particularly, the low-pass filter version:

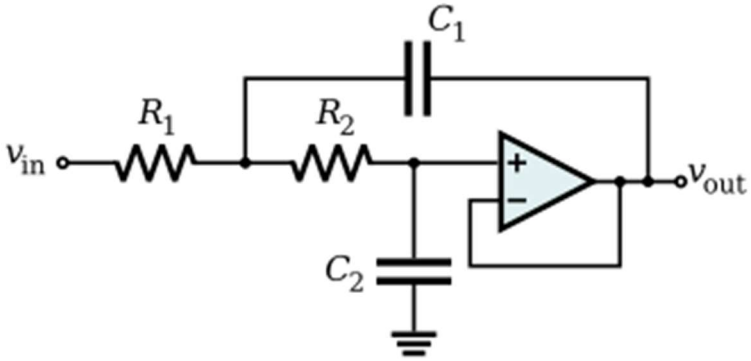


Figure 6: Standard Sallen-Key Low-pass Filter

$$\frac{V_o}{V_i} = \frac{w_o^2}{s^2 + 2\alpha s + w_o^2} = \frac{\frac{1}{R_1 R_2 C_1 C_2}}{s^2 + \frac{R_1 + R_2}{R_1 R_2 C_2} s + \frac{1}{R_1 R_2 C_1 C_2}}$$

Where $\frac{R_1+R_2}{R_1 R_2 C_2} = 2a$, and $\frac{1}{R_1 R_2 C_1 C_2} = w_o^2$, where w_o is the frequency that causes 0 output (shorts capacitors, virtual ground.)

Transfer Function Derivation

Given:

- Z1, Z2 = R1, R2
- Z3, Z4 = C1, C2

Stage 1:

$$H(s) = \frac{V_o}{V_i} = \frac{10375^2}{s^2 + 34662s + 10375^2}$$

$$\frac{10375}{2\pi} = \sim 1651 \cong w_o$$

Stage 2:

$$H(s) = \frac{V_o}{V_i} = \frac{10373^2}{s^2 + 17094s + 10373^2}$$

$$\frac{10373}{2\pi} = \sim 1651 \cong w_o$$

<u>Loads</u>

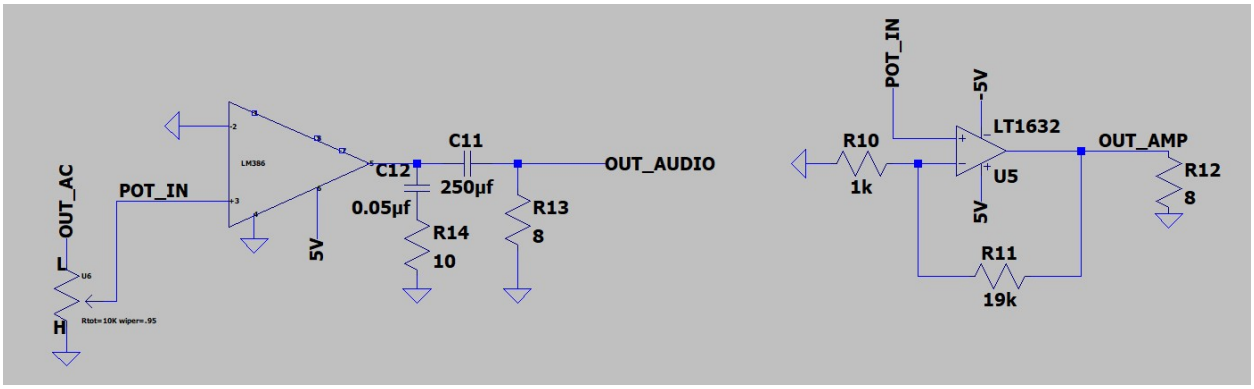


Figure 7: Schematics for op-amp circuit and LM386 audio amp.

Results

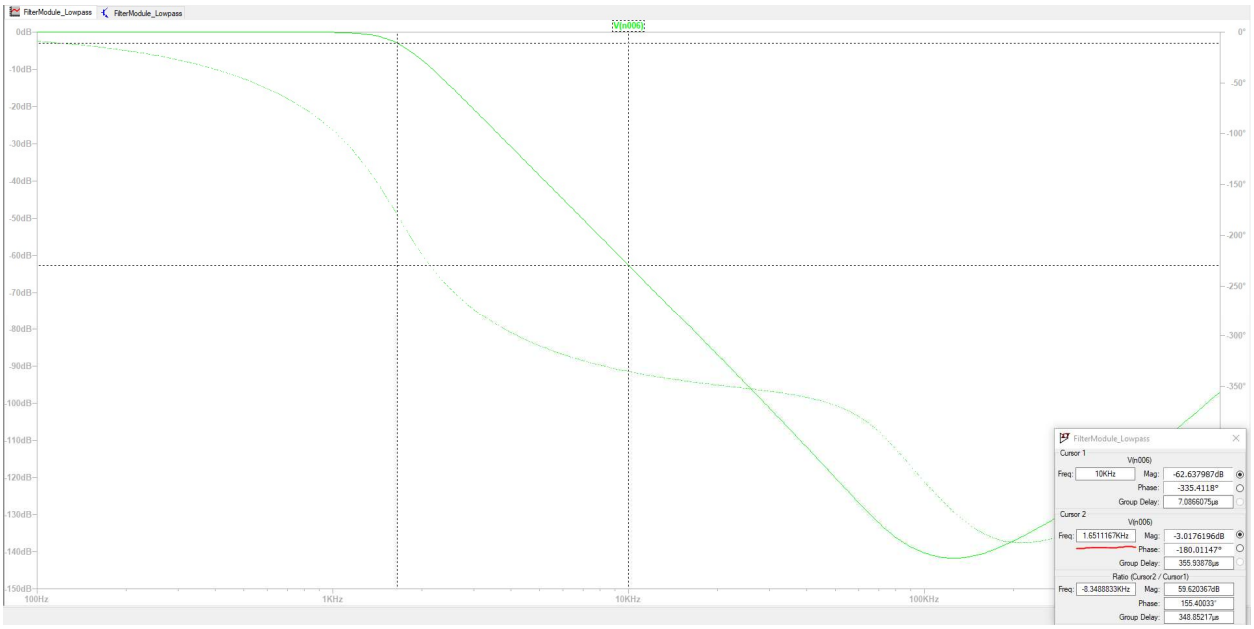


Figure 8: Bode (AC) Analysis of Simulated Filter

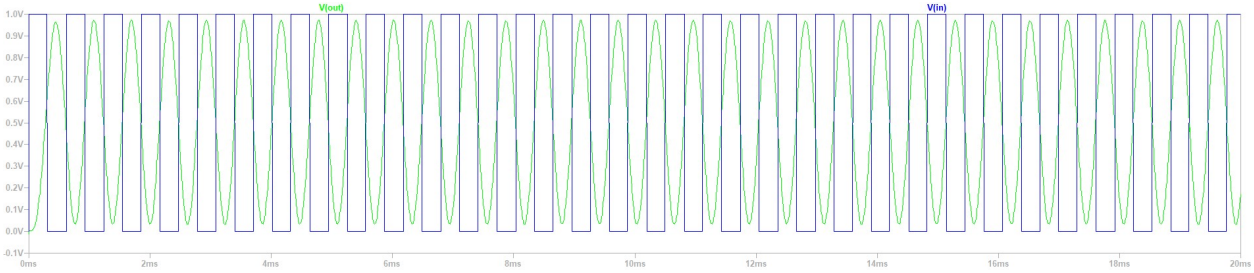


Figure 9: Transient Analysis of Simulated Filter (0ms -> 20ms)

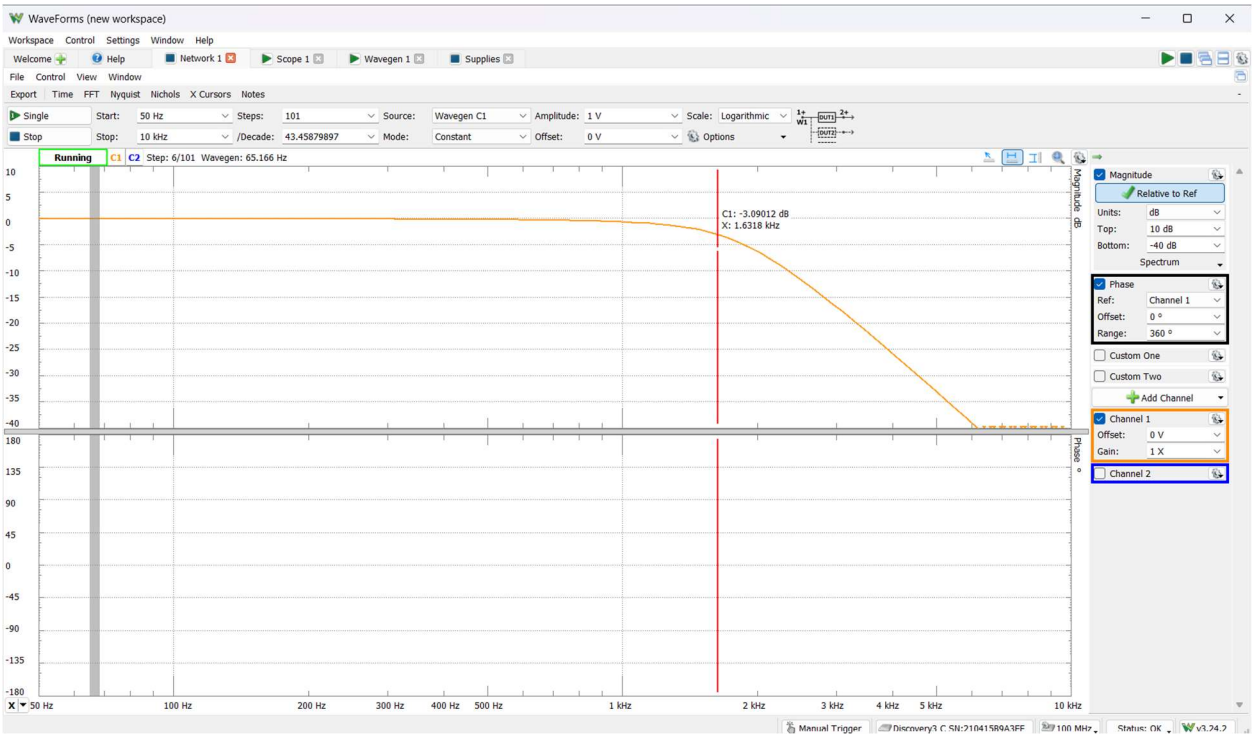


Figure 10: Bode (AC) Analysis of Physical Filter

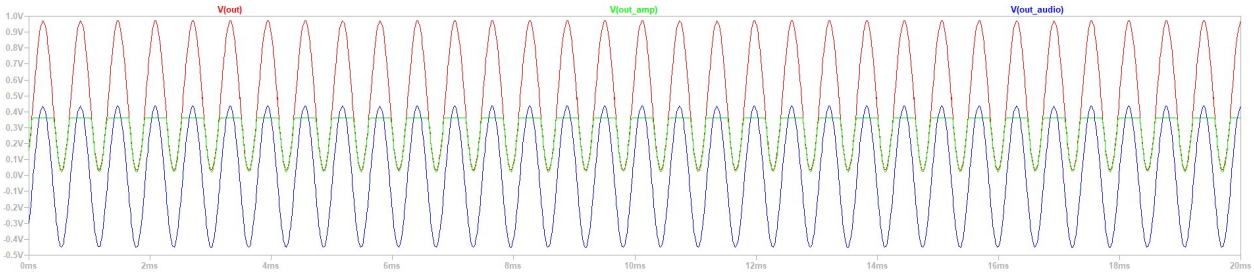


Figure 11: Transient Analysis of Physical Amplifier (20ms -> 40ms)

Results Analysis

The results show that the cutoff frequency drifted to a lower frequency than expected, which is likely due to the resistor availability & tolerances (+-5%), and the limitation of availability of precision capacitors (such as .13uF).

The overall form, being a Butterworth filter, remained, as we had a smooth passband with a steady dropoff to the stopband. To bring the cutoff frequency to a point where it prevented the attenuation of the sinusoidal wave output, I had to raise the cutoff frequency slightly from the initial design, by using slightly smaller resistors to raise the fundamental frequency.

On our output simulation of the amplifier, we can see that the LM386 has allows for a larger swing voltage, as it gives itself a DC bias to center close to 0V, and provides external power to drive higher currents, where the op amp clips (reading as a flat peak.)

Vpp	Power Delivered (Vpp^2)/8
350mV	15mW
850mV	90mW

The LM386 offers greater power supply, which allows it to have larger Vpp, but uses more energy. The op amp will quickly run out of power relatively, due to current limitation. The LT1632 is better for precision signal amplification since it is designed for linear operation, but it can not supply significant power to a load. The LM386 is better in its power delivery, but has worse precision compared to the LT1632. The op-amp IC that was chosen was the LT1632 due to having great precision, but due to it's slower slew rate, it did struggle to swing from rail to rail at times, this was fixed by provided +-5V instead of 3.3V to improve slew rate.

Conclusion

The LM386 provides a higher output amplification than that of the LT1632, it can deliver more power, and centers itself on 0V DC, so you can easily read the output swing. The LT1632 can be great for precision filtering, but not as well for amplification, it is also better to operate at higher voltage rails to improve performance when possible. Having an audio amplifier is important when driving low resistance loads as the current will drive inversely proportional to the load it is serving.

References

(1) Wikipedia contributors. (2026, March 13). *Sallen–Key topology*. Wikipedia. https://en.wikipedia.org/wiki/Sallen%E2%80%93Key_topology

(2) https://tools.analog.com/en/filterwizard/

(3) AI Utilized: ChatGPT for debugging plotting and filtering.

APPENDIX

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Fri Jan  7 19:02:05 2022
Design 1 Analog/Digital Filter Module:
    Understand mathematical filter functions (Butterworth, Bessel, Chebyshev)
    Uses scipy.signal package for analog and digital filter design
@author: erin patrick
"""
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt
from scipy.fftpack import fft
from scipy.fftpack import fftfreq
from scipy.fftpack import ifft



#--------Time domain signals / Input waveforms---------
fs = 100e3  #sampling frequency samples/s (note one can only plot frequencies up to 1/2fs,
i.e. Nyquist frequency)
dt = 1.0/fs #time step (s)
end = 0.01  #length of time signal (s)
N = int(end*fs)   #number of samples
t = np.linspace(0,end,N)   #array of time points of quantity N (s)

##define the square wave using signal.square
fsq = 1619   #repetition frequency of square wave (Hz)
square_sig=signal.square(2 * np.pi * fsq * t)  # duty cycle defaults to 0.5

# add any other waveforms you wish to explore (e.g., sine, sine with broadband noise, etc.)


#-----------Filters---------------------
#Specs for lowpass filter

wc2= 2*np.pi*(fsq+1650)  # cut off frequency (rad/s), Change this accordingly
order = 4 #filter order (1=20 dB/decade rolloff, 2 = 40 dB/dec, etc.)
fplot = np.logspace(1, 4, N)  # (specify frequency range of interest, 2=10e2, 4=10e4)

#  Butterworth filter
b, a = signal.butter(order, wc2, btype='low', analog = True)
w, h = signal.freqs(b, a, worN=fplot*2*np.pi)

#  Bessel
b2, a2 = signal.bessel(order,  wc2 ,'lowpass', analog=True)
w2, h2 = signal.freqs(b2, a2, worN=fplot*2*np.pi)

#   Chebyshev
rp = 1  #ripple in pass pand
b3, a3 = signal.cheby1(order, rp, wc2 , 'lowpass', analog=True)
w3, h3 = signal.freqs(b3, a3, worN=fplot*2*np.pi)

#-------------------------------------------------



#-----------Plot the fitler response and time-domain filtered signal------

#plot the frequency response of the analog filters

plt.figure(1)
plt.subplot(2,1,1)
plt.title('Aanlog Filter')
```

```
plt.semilogx(w/(2*np.pi), 20 * np.log10(abs(h)), "b", w2/(2*np.pi), 20 * np.log10(abs(h2)),
'--r', w3/(2*np.pi), 20 * np.log10(abs(h3)),'g')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Analog Filter Gain (dB)')
plt.margins(0, 0.1)
plt.grid(which='both', axis='both')
plt.axvline(fsq, color='k') # square wave frequency

# Compute FFT of time-domain signals
fft_square = fft(square_sig) #complex value of the FFT spectrum
freq_pulse = fftfreq(len(t), d=dt) #frequnecy array, pass in length of signal and step size

# compute a transfer function of a filter type using the same frequencies as the FFT of the
time signal
w, h = signal.freqs(b, a, worN=freq_pulse*2*np.pi) # change filter type as needed
w2, h2 = signal.freqs(b2, a2, worN=freq_pulse*2*np.pi) # change filter type as needed
(i.e., a,b go to a Butterworth filter. , a2,b2 go to a Bessel, etc.)
w3, h3= signal.freqs(b3, a3, worN=freq_pulse*2*np.pi)

#Compute an inverseFFT of the filtered signal (multipy square FFT by filter transfer
function )
filteredt1 = ifft(fft_square*h)
filteredt2 = ifft(fft_square*h2)
filteredt3 = ifft(fft_square*h3)


#plot square wave and filtered time domain wave

plt.subplot(2,1,2)

plt.plot(t, square_sig, '-k', t, (filteredt1), '-b', t, filteredt2, '--r', t, filteredt3,
'-g')
plt.legend(['original','filtered-Butterworth','filtered-Bessel','filtered-Chebyshev'])
plt.xlabel('time (s)')
plt.ylabel('Voltage (V)')

plt.legend(['Butterworth', 'Bessel', 'Chebyshev'])
plt.show()

#-----------Plot the frequency-domain filtered signal------

# compute a transfer function of a filter type using the same frequencies as the FFT of the
time signal
#w, h = signal.freqs(b, a, worN=freq_pulse*2*np.pi) # change filter type as needed (i.e.,
a,b go to a Butterworth filter. , a2,b2 go to a Bessel, etc.)
#w, h2 = signal.freqs(b2, a2, worN=freq_pulse*2*np.pi) # change filter type as needed
(i.e., a,b go to a Butterworth filter. , a2,b2 go to a Bessel, etc.)
#w, h3= signal.freqs(b3, a3, worN=freq_pulse*2*np.pi) # change filter type as needed (i.e.,
a,b go to a Butterworth filter. , a2,b2 go to a Bessel, etc.)
# Multiply the FFT of the square wave by the transfer function of the filter to get the
filterred signal
filtered1 =2.0/N *
np.abs(fft_square[0:int(len(freq_pulse)/2)]*(h[0:int(len(freq_pulse)/2)]))  #only take 1/2
of data since it is mirrored and scale by factor to get correct output
filtered2 =2.0/N *
np.abs(fft_square[0:int(len(freq_pulse)/2)]*(h2[0:int(len(freq_pulse)/2)]))
filtered3 =2.0/N *
np.abs(fft_square[0:int(len(freq_pulse)/2)]*(h3[0:int(len(freq_pulse)/2)]))



plt.figure(3)
# plot the FFT of the original signal
plt.semilogx(freq_pulse[0:int(len(freq_pulse)/2)], 2.0/N *
np.abs(fft_square[0:int(len(freq_pulse)/2)]), '-k')
#plot the frequency response of the filtered signal
plt.semilogx(freq_pulse[0:int(len(freq_pulse)/2)], filtered1 , '-b',
```

```
freq_pulse[0:int(len(freq_pulse)/2)], filtered2 , '--
r',freq_pulse[0:int(len(freq_pulse)/2)], filtered3 , '-g')
plt.xlabel('frequency (Hz)')
plt.ylabel('FFT Magnitude')
plt.legend(['original','filtered-Butterworth','filtered-Bessel','filtered-Chebyshev' ])
plt.title('Signal Frequency Response (analog)')


#----------Digital Filter Section-----------------
rate = 10000 #(samples/s, rate defined in the DMA Python code)
order = 4
x = np.linspace(0, (1023*1/rate), 1023)
y = signal.square(2*np.pi*fsq*x)+1.0
f_cutoff = fsq+00 #(Hz)
w_cutoff = f_cutoff*2*np.pi
sos_cheb = signal.cheby1(order, rp, f_cutoff, 'lowpass', fs=rate, output='sos')
sos_butter = signal.butter(order, f_cutoff, 'lowpass', fs=rate, output='sos')
sos_bessel = signal.bessel(order, f_cutoff, 'lowpass', fs=rate, output='sos')
sosfilt_cheb = signal.sosfilt(sos_cheb, y)
sosfilt_butter = signal.sosfilt(sos_butter, y)
sosfilt_bessel = signal.sosfilt(sos_bessel, y)

plt.figure(4)
plt.subplot(2,1,2)
plt.plot(x,y, 'k', x, sosfilt_butter, '-b', x, sosfilt_bessel, '--r', x, sosfilt_cheb, '-
g')
plt.xlabel('Time (s)')
plt.ylabel('Time Signal (V)')
plt.legend(['original','filtered-Butterworth','filtered-Bessel','filtered-Chebyshev'])
plt.show()

#plt.figure(5)
w, h_butter = signal.sosfreqz(sos_butter)
w, h_bessel = signal.sosfreqz(sos_bessel)
w, h_cheb = signal.sosfreqz(sos_cheb)
db_butter = 20*np.log10(np.maximum(np.abs(h_butter), 1e-5))
db_bessel = 20*np.log10(np.maximum(np.abs(h_bessel), 1e-5))
db_cheb = 20*np.log10(np.maximum(np.abs(h_cheb), 1e-5))

plt.subplot(2,1,1)
plt.title('Digital Filter')
plt.semilogx(w/np.pi*rate/2, db_butter,'-b', w/np.pi*rate/2, db_bessel,'--r',
w/np.pi*rate/2, db_cheb, '-g')
plt.grid(which='both', axis='both')
plt.xlabel('frequency (Hz)')
plt.ylabel('Digital Filter Gain (dB)')
plt.axvline(fsq, color='k') # square wave frequency
plt.legend(['Butterworth', 'Bessel', 'Chebyshev'])

plt.show()
```